

Smart Campus University Portal

Assignment Report — JDBC and Servlets

Subject: Programming in JAVA

Name: Jehkaran Singh	SAP: 590019196
Mail: jeh.19196@stu.upes.ac.in	Course: Btech. Electronics and Computer Engineering 3rd Year

1. Objective

The objective of this assignment is to develop a dynamic web application called the **Smart Campus University Portal** using Java Servlets, JDBC, and Java Collections Framework. The application demonstrates:

- Connecting a Java web application to a MySQL database using JDBC
- Performing CRUD operations (Create, Read, Update, Delete) through Servlets
- Using `PreparedStatement` for secure, parameterized SQL queries
- Mapping database results into Java Collection objects (`ArrayList` , `HashSet` , `HashMap` , `LinkedList`)
- Deploying the application on Apache Tomcat 9 as a Dynamic Web Project in Eclipse IDE

2. Database Schema

Database Name: `campus_portal`

Table 1: `users`

Column	Data Type	Constraint	Description
user_id	INT	PRIMARY KEY	Unique identifier for each user
name	VARCHAR(50)	NOT NULL	Full name of the user
email	VARCHAR(100)	NOT NULL	Email address of the user
role	VARCHAR(30)	NOT NULL	Role: Student / Faculty / Administrator

Table 2: `service_request`

Column	Data Type	Constraint	Description
request_id	INT	PRIMARY KEY	Unique identifier for each request
user_id	INT	FOREIGN KEY → users	References the user who made the request
service_name	VARCHAR(50)	NOT NULL	Type of campus service requested
request_details	VARCHAR(200)	—	Description of the request
status	VARCHAR(30)	DEFAULT 'Pending'	Current status of the request

SQL Creation Script

```
CREATE DATABASE IF NOT EXISTS campus_portal;
USE campus_portal;
```

```
CREATE TABLE IF NOT EXISTS users (
    user_id INT PRIMARY KEY,
    name    VARCHAR(50) NOT NULL,
    email   VARCHAR(100) NOT NULL,
    role    VARCHAR(30) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS service_request (
    request_id INT PRIMARY KEY,
    user_id    INT NOT NULL,
    service_name VARCHAR(50) NOT NULL,
    request_details VARCHAR(200),
    status     VARCHAR(30) NOT NULL DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Screenshot — Table Creation

	user_id	name	email	role
▶	101	Jehkaran Singh	jeh@upes.ac.in	Student
	102	Srishti	srish@upes.ac.in	Faculty
	103	Sneha	sneha@upes.ac.in	Student

Screenshot – Table Structure

	Field	Type	Null	Key	Default	Extra
▶	user_id	int	NO	PRI	NULL	
	name	varchar(50)	NO		NULL	
	email	varchar(100)	NO		NULL	
	role	varchar(30)	NO		NULL	

	request_id	user_id	service_name	request_details	status
▶	1	101	Transport	Bus pass renewal	Pending
	2	101	Library	Book reservation request	Approved
	3	102	IT Support	Projector not working	Pending
	4	103	Hostel	Room change request	Processing
	5	101	Complaint Desk	Canteen food quality	Pending
	6	103	Lab	Equipment booking	Completed

3. Servlet List and Purpose

The application contains **8 Servlets**, all placed in the package `com.campus.servlet`.

Servlet Class	URL Mapping	HTTP Method	Purpose
<code>RegisterUserServlet</code>	<code>/RegisterUserServlet</code>	POST	Inserts a new user record into the <code>users</code> table
<code>ServiceRequestServlet</code>	<code>/ServiceRequestServlet</code>	POST	Inserts a new service request into the <code>service_request</code> table
<code>ViewRequestsServlet</code>	<code>/ViewRequestsServlet</code>	GET	Fetches all service requests and stores them in an <code>ArrayList</code>

Servlet Class	URL Mapping	HTTP Method	Purpose
<code>UniqueServicesServlet</code>	<code>/UniqueServicesServlet</code>	GET	Fetches all service names and stores unique values in a <code>HashSet</code>
<code>UserRequestSummaryServlet</code>	<code>/UserRequestSummaryServlet</code>	GET	Counts requests per user using a <code>HashMap</code>
<code>RequestQueueServlet</code>	<code>/RequestQueueServlet</code>	GET	Fetches all Pending requests and stores them in a <code>LinkedList</code>
<code>UpdateStatusServlet</code>	<code>/UpdateStatusServlet</code>	POST	Updates the status of an existing service request
<code>DeleteRequestServlet</code>	<code>/DeleteRequestServlet</code>	POST	Deletes a service request record by request ID

4. HTML Page List and Purpose

All HTML pages are placed in `src/main/webapp/`.

HTML File	Purpose
<code>index.html</code>	Home page with navigation links to all features
<code>register.html</code>	Form to register a new user (User ID, Name, Email, Role)
<code>request.html</code>	Form to submit a new service request
<code>update.html</code>	Form to update the status of an existing service request
<code>delete.html</code>	Form to delete a service request by Request ID

5. JDBC Operations Used

5.1 JDBC Connection Flow

JDBC (Java Database Connectivity) is the standard Java API for connecting to relational databases. The connection flow in this project is:

```

Java Application
  ↓
Class.forName("com.mysql.cj.jdbc.Driver") ← Load the MySQL driver
  ↓
DriverManager.getConnection(URL, USER, PASS) ← Establish connection
  ↓
Connection object
  ↓
PreparedStatement ← Prepare SQL with placeholders
  ↓
executeQuery() or executeUpdate() ← Execute the SQL
  ↓
ResultSet (for SELECT) ← Iterate over results
  ↓
Connection closed (try-with-resources)

```

5.2 DBConnection Utility Class

A centralized `DBConnection` class is used to avoid repeating connection code in every servlet:

```

public class DBConnection {
    private static final String URL      = "jdbc:mysql://localhost:3306/campus_portal";
    private static final String USER     = "root";
    private static final String PASSWORD = "root";

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new SQLException("MySQL JDBC Driver not found.",
e);
        }
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

Every servlet calls `DBConnection.getConnection()` to obtain a connection. The connection is wrapped in a `try-with-resources` block so it is automatically closed after use, preventing resource leaks.

5.3 PreparedStatement

`PreparedStatement` is used instead of `Statement` for all SQL operations. It accepts `?` placeholders which are filled using setter methods (`setInt`, `setString`, etc.) before execution.

Advantages over `Statement` :

- Prevents SQL Injection — user input is never directly embedded in the SQL string
- Improves performance — the query is pre-compiled by the database
- Cleaner code — parameters are set by position, not by string concatenation

Example — inserting a user:

```
String sql = "INSERT INTO users (user_id, name, email, role) VALUES (?, ?, ?, ?)";

PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, userId);           // replaces first ?
ps.setString(2, name);          // replaces second ?
ps.setString(3, email);         // replaces third ?
ps.setString(4, role);          // replaces fourth ?

int rows = ps.executeUpdate();
```

5.4 ResultSet

`ResultSet` is the object returned by `executeQuery()`. It acts as a cursor that starts before the first row. Calling `rs.next()` moves it forward one row at a time, returning `false` when there are no more rows.

Example — reading rows:

```
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    int id = rs.getInt("request_id");
    String name = rs.getString("service_name");
    String status = rs.getString("status");
}
```

5.5 executeQuery() vs executeUpdate()

Method	Used For	Returns	Example Operations
<code>executeQuery()</code>	Statements that return data	<code>ResultSet</code> object	SELECT
<code>executeUpdate()</code>	Statements that modify data	<code>int</code> (number of rows affected)	INSERT, UPDATE, DELETE

executeQuery() — used in read servlets:

```
// Returns a ResultSet containing matching rows
ResultSet rs = ps.executeQuery();
```

executeUpdate() — used in write servlets:

```
// Returns the number of rows affected (0 = nothing changed, 1+ = success)
int rows = ps.executeUpdate();

if (rows > 0) {
    out.println("Operation successful.");
}
```

In `DeleteRequestServlet`, the return value of `executeUpdate()` is printed explicitly to show how many records were deleted, demonstrating that the method returns a row count.

6. Collections Used

6.1 ArrayList — ViewRequestsServlet

`ArrayList<ServiceRequest>` is used to store all service requests fetched from the database before displaying them.

Why ArrayList: Preserves insertion order, allows indexed access, and stores the full `ServiceRequest` object rather than raw strings. The `ResultSet` is converted to an `ArrayList` so the database connection can be closed before the data is rendered — a better separation of concerns.

```
ArrayList<ServiceRequest> requestList = new ArrayList<>();

while (rs.next()) {
```

```

        requestList.add(new ServiceRequest(
            rs.getInt("request_id"),
            rs.getInt("user_id"),
            rs.getString("service_name"),
            rs.getString("request_details"),
            rs.getString("status")
        ));
    }

    for (ServiceRequest sr : requestList) {
        out.println("<tr><td>" + sr.getRequestId() + "</td>...</tr>");
    }

```

Key characteristics: Ordered, indexed, allows duplicates, backed by a dynamic array.

6.2 HashSet — UniqueServicesServlet

`HashSet<String>` is used to collect all distinct service names from the `service_request` table.

Why HashSet: A HashSet automatically rejects duplicate values. If “Transport” appears in 10 rows, the set will only hold it once — no manual duplication check is required.

```

HashSet<String> uniqueServices = new HashSet<>();

while (rs.next()) {
    uniqueServices.add(rs.getString("service_name")); // duplicate
    s ignored
}

for (String service : uniqueServices) {
    out.println("<li>" + service + "</li>");
}

```

Key characteristics: Unordered, no duplicates, uses `hashCode()` and `equals()` internally. Output order is not guaranteed.

6.3 HashMap — UserRequestSummaryServlet

`HashMap<Integer, Integer>` is used to count how many service requests each user has submitted. The key is the `user_id` and the value is the running count.

Why HashMap: Provides O(1) lookup by key. `getOrDefault(key, 0)` elegantly handles the first occurrence of a user ID without needing a null check.

```
HashMap<Integer, Integer> userRequestCount = new HashMap<>();

while (rs.next()) {
    int uid = rs.getInt("user_id");
    userRequestCount.put(uid, userRequestCount.getOrDefault(uid,
0) + 1);
}

for (Map.Entry<Integer, Integer> entry : userRequestCount.entrySet
()) {
    out.println("User " + entry.getKey() + ": " + entry.getValue()
+ " requests");
}
```

Key characteristics: Key-value pairs, keys are unique, unordered, allows null values.

6.4 LinkedList — RequestQueueServlet

`LinkedList<ServiceRequest>` is used to model a service queue. Only requests with `status = 'Pending'` are fetched and stored in the queue. `peek()` returns the front element (the next request to be processed) without removing it.

Why LinkedList: Implements the `Queue` interface natively. `peek()`, `poll()`, and `add()` are all queue operations that communicate intent clearly — this is a processing queue, not just a display list.

```
LinkedList<ServiceRequest> pendingQueue = new LinkedList<>();

while (rs.next()) {
    pendingQueue.add(new ServiceRequest(...)); // enqueue
}

ServiceRequest next = pendingQueue.peek(); // look at front without removing
out.println("Next to process: Request ID " + next.getRequestId());
```

Key characteristics: Doubly-linked, ordered (FIFO), allows duplicates, efficient insertion and removal at both ends.

7. Servlet Flow Explanation

When a user submits a form or clicks a link, the following sequence occurs:

POST Flow (e.g., Register User)

1. User fills register.html and clicks Submit
2. Browser sends HTTP POST to /SmartCampusPortal/RegisterUserServlet
3. Tomcat matches the URL to @WebServlet("/RegisterUserServlet")
4. Tomcat calls doPost(HttpServletRequest, HttpServletResponse)
5. Servlet reads form parameters via request.getParameter("user_id") etc.
6. Servlet calls DBConnection.getConnection() to get a Connection
7. PreparedStatement is created with the INSERT SQL
8. Parameters are bound using ps.setInt(), ps.setString()
9. ps.executeUpdate() sends the SQL to MySQL
10. MySQL executes the INSERT and returns the row count
11. Servlet writes the result to response via PrintWriter out
12. Browser displays the response HTML
13. Connection is closed automatically (try-with-resources)

GET Flow (e.g., View All Requests)

1. User clicks "View All Requests" link
2. Browser sends HTTP GET to /SmartCampusPortal/ViewRequestsServlet
3. Tomcat calls doGet(HttpServletRequest, HttpServletResponse)
4. Servlet opens DB connection and executes SELECT via executeQuery()
5. ResultSet rows are converted into ServiceRequest objects
6. Each object is added to an ArrayList<ServiceRequest>
7. DB connection is closed
8. Servlet iterates the ArrayList and writes an HTML table to the response
9. Browser renders the table

8. Screenshots of Output

8.1 Database Setup

	user_id	name	email	role
▶	101	Jehkaran Singh	jeh@upes.ac.in	Student
	102	Srishti	srish@upes.ac.in	Faculty
	103	Sneha	sneha@upes.ac.in	Student

	request_id	user_id	service_name	request_details	status
▶	1	101	Transport	Bus pass renewal	Pending
	2	101	Library	Book reservation request	Approved
	3	102	IT Support	Projector not working	Pending
	4	103	Hostel	Room change request	Processing
	5	101	Complaint Desk	Canteen food quality	Pending
	6	103	Lab	Equipment booking	Completed

8.2 Application Running

Smart Campus University Portal

Welcome! Use the links below to navigate the portal.

[Register User](#)[Submit Service Request](#)[View All Requests \(ArrayList\)](#)[View Unique Services \(HashSet\)](#)[View User Request Summary \(HashMap\)](#)[View Pending Request Queue \(LinkedList\)](#)[Update Request Status](#)[Delete Request](#)

localhost:8080/SmartCampusPortal/index.html

8.3 Register User

Register User

User ID:

Name:

Email:

Role:



[Back to Home](#)



localhost:8080/SmartCampusPortal/register.html

Register User

User ID:

Name:

Email:

Role:



[Back to Home](#)



localhost:8080/SmartCampusPortal/register.html

8.4 Submit Service Request

Submit Service Request

Request ID:

User ID:

Service Name:

Request Details:

Status:

[Back to Home](#)

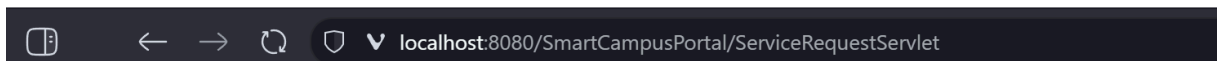


localhost:8080/SmartCampusPortal/request.html

Service Request Submission

Service request submitted successfully! Request ID: 7

[Back to Home](#)



8.5 View All Requests (ArrayList)

All Service Requests

Request ID	User ID	Service	Details	Status
1	101	Transport	Bus pass renewal	Pending
2	101	Library	Book reservation request	Approved
3	102	IT Support	Projector not working	Pending
4	103	Hostel	Room change request	Processing
5	101	Complaint Desk	Canteen food quality	Pending
6	103	Lab	Equipment booking	Completed
7	102	Library	Book Reissue	Processing

Total records in ArrayList: 7

[Back to Home](#)

localhost:8080/SmartCampusPortal/ViewRequestsServlet

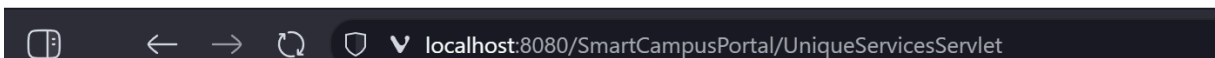
8.6 Unique Services (HashSet)

Unique Campus Services

- Hostel
- Library
- Complaint Desk
- Transport
- IT Support
- Lab

Total unique services (HashSet size): 6

[Back to Home](#)



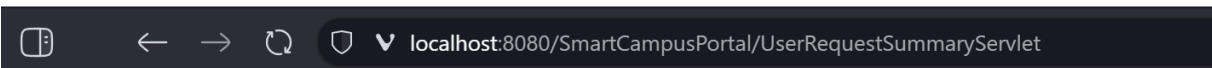
8.7 User Request Summary (HashMap)

User Request Summary

User ID	Number of Requests
101	3
102	2
103	2

Total users in HashMap: 3

[Back to Home](#)



8.8 Pending Request Queue (LinkedList)

Pending Request Queue

Next to Process: Request ID 1 ? Transport

ID	User	Service	Details	Status
1	101	Transport	Bus pass renewal	Pending
3	102	IT Support	Projector not working	Pending
5	101	Complaint Desk	Canteen food quality	Pending

Total in LinkedList: 3

[Back to Home](#)



8.9 Update Status

Update Request Status

Request ID:

New Status:



Update Status

[Back to Home](#)

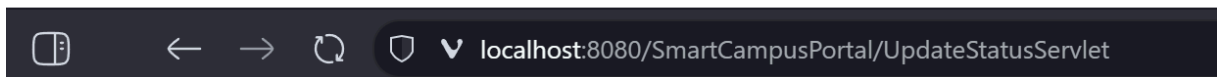


localhost:8080/SmartCampusPortal/update.html

Update Request Status

Request ID 1 status updated to: **Approved**

[Back to Home](#)



8.10 Delete Request

Delete Service Request

Request ID to Delete:

Delete Request

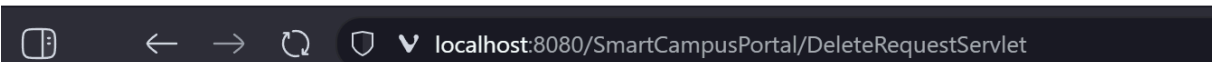
[Back to Home](#)

Delete Request

Number of records deleted: 1

Request ID 6 deleted successfully.

[Back to Home](#)



9. Conclusion

This assignment successfully demonstrates the integration of Java Servlets with a MySQL database using JDBC in a Dynamic Web Project deployed on Apache Tomcat 9. The key learning outcomes are:

- **JDBC** provides a clean, standardized API for database connectivity in Java
 - **PreparedStatement** is essential for security (SQL injection prevention) and code clarity
 - **executeQuery()** and **executeUpdate()** serve distinct purposes — reads vs. writes
 - **ResultSet** acts as a database cursor and must be converted to in-memory collections before the connection is closed
 - Each **Java Collection** (`ArrayList` , `HashSet` , `HashMap` , `LinkedList`) serves a specific structural purpose that maps naturally to real-world data requirements
 - The `@WebServlet` annotation eliminates the need for verbose XML servlet mappings
 - Separating the DB utility into `DBConnection.java` follows the principle of separation of concerns and avoids code duplication
-